

Algorithmique et Programmation orientée-objet

Ch.3 – Tableaux

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Objectifs

- **Les objectifs de ce chapitre sont**
 - Introduire le type tableau en Java
 - Déclarer, instancier un tableau
 - Accéder aux éléments d'un tableau
 - Contraintes d'utilisation : taille fixe, vérification de type, vérification des bornes
 - Copie de tableaux
 - Algorithmes sur les tableaux
 - Boucles for améliorée
 - Tableaux à plusieurs dimensions

Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

Tableaux

• Exemple introductif

- Un tableau en Java est une séquence d'éléments de même type et de longueur fixe.

- Exemples

Type
tableau
d'int

```
int[] boTablo = { 1, 2, 3, 5, 7, 11 };
```

```
System.out.println(boTablo[2]);  
boTablo[2] = 17;  
System.out.println(boTablo.length);
```

Initialisation

boTablo ref

Accès

longueur

:int[]	
0	1
1	2
2	3
3	5
4	7
5	11

Type
tableau
de Carre

```
Carre[] carres = {  
    new Carre(4, 1, 0, 2),  
    new Carre(5, 7, -60, 5)  
};  
carres[1].tourner(10);
```



tableau d'objets
=
tableau de références

carres ref

:Carre[]	
0	ref
1	ref

:Carre	
x	5
y	7
angle	-60
cote	5

:Carre	
x	4
y	1
angle	0
cote	2

Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

Tableaux

- **Type tableau**

- `T[]` désigne le **type tableau** d'éléments de type `T` où `T` peut être *primitif* (`int`, `double`, ...) ou *objet* (`String`, `Carre`, ...)
- Comme tout autre type, le type tableau peut apparaître dans les déclarations de variables locales, d'attributs de classe et d'instance, et dans les arguments de méthodes.
- Deux types tableaux `X[]` et `Y[]` sont compatibles ssi `X` et `Y` sont compatibles (en particulier si `X = Y`).

Note : la longueur du tableau ne fait pas partie du type

Tableaux

- **Déclaration**

- La déclaration d'une variable tableau comprend le type tableau, un nom et une initialisation optionnelle.

```
nomType[] nomVariable [ = initialisation ] ;
```

- Un tableau est un **objet**. ⇒ Une variable tableau stocke donc une **référence**. Il faut instancier un tableau avant de l'utiliser !!

```
double[] tab;
```



- **Convention de nommage**

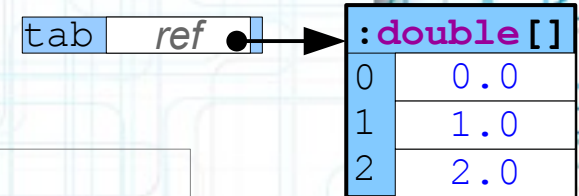
- Les noms de variables de type tableau respectent la convention des variables (minuscules + « chameau »).
- Ils sont préférentiellement **au pluriel** et peuvent comporter des termes tels que « Tableau » (« Array ») ou « Elements » (« Items »)

Tableaux

- **Initialisation et instantiation**

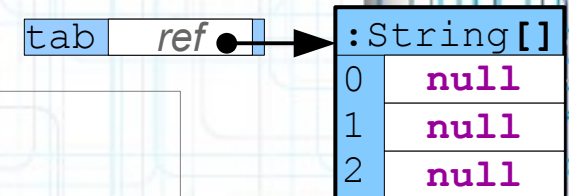
- **Initialisation** – uniquement lors de la déclaration

```
double[] tab = { 0.0, 1.0, 2.0 };
```



- **Instantiation** – utilisation opérateur **new**

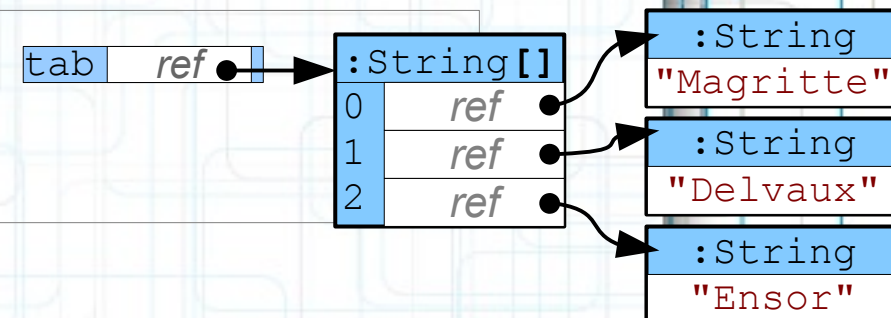
```
String[] tab;  
tab = new String[3];
```



- cellules initialisées avec valeur par défaut, en fonction de leur type (0 pour primitifs numériques, **null** pour objets)

- **Instantiation avec initialisation**

```
String[] tab;  
tab = new String[] {  
    "Magritte", "Ensor", "Delvaux"  
};
```



Tableaux

Note : il n'existe pas en Java d'opérateur de *slicing* comme en Python. En revanche, il est possible d'utiliser les méthodes `CopyOfRange` de la classe `java.util.Arrays`

- **Indexation d'un tableau**

- L'opérateur d'indexation `tab[i]` permet d'accéder aux éléments d'un tableau. C'est un opérateur binaire qui prend la référence d'un tableau et un index entier.
 - L'élément résultant peut être utilisé dans une affectation (*left-value*) ou dans une expression (*right-value*).
 - L'index est une expression entière dont la valeur doit être comprise entre 0 et `length-1`. L'accès en dehors de ces bornes déclenche `ArrayIndexOutOfBoundsException`.

- Exemple

```
int[] tab = { 1, 2, 3 };  
System.out.println(tab[0]);  
tab[2] += 1;  
tab[3] = 5;           // ArrayIndexOutOfBoundsException
```

Tableaux

- **Boucle for-each**

- La boucle **for-each** (ou *enhanced-for*) permet de parcourir séquentiellement l'ensemble des éléments d'un tableau. Elle permet de rendre le code plus compact et plus lisible.

- Syntaxe

```
for ( nomType nomVariable : tableau )  
    bloc
```

- Exemple

```
char[] tab = { 'f', 'o', 'r' };  
String msg= "";  
for (int i= 0; i < tab.length; i++)  
    msg = msg + tab[i];
```



```
char[] tab = { 'f', 'o', 'r' };  
String msg= "";  
for (char c : tab)  
    msg = msg + c;
```


Tableaux

- Exemple complet

```
final int MAX_VALUES = 10;

Scanner scan = new Scanner(System.in);
int[] tab = new int[MAX_VALUES];
int numValues = 0;

do {
    int val = scan.nextInt();
    if (val == 0)
        break;
    tab[numValues] = val;
    numValues++;
} while (numValues < MAX_VALUES);

int sum = 0;
for (int i: = 0; i < numValues; i++)
    sum += tab[i];
double moyenne = ((1.0 * sum) / numValues)

System.out.println("Moyenne = " + moyenne);
```


Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

Applications particulières

- **Arguments du programme**

- Les arguments passés en ligne de commande à un programme Java sont passés dans un tableau de `String` à la méthode `main`.
- Exemple

```
public static void main(String[] args) {  
    int index = 0;  
    while (index < args.length) {  
        System.out.println("Arg[" + index + "]: \"" +  
                           args[index] + "\"");  
        index++;  
    }  
}
```

```
bash-3.2$ java ArgsExample 5 "coco est content"  
Arg[0]: "5"  
Arg[1]: "coco est content"  
bash-3.2$
```

Applications particulières

- **Méthode à nombre variable d'arguments**

- Il est possible de définir une méthode prenant un **nombre variable d'arguments** (*vararg*). Ces arguments sont passés à la méthode sous forme d'un tableau.
- Exemple

```
public static void methode(int param1, String... varargs) {
    System.out.println("Param 1 = "+param1);
    for (int index = 0; index < varargs.length; index++)
        System.out.println("Param "+(index+1)+" = "+varargs[index]);
}
```

```
methode(1, "coucou", "coco");
```

```
Param 1 = 1
Param 2 = coucou
Param 3 = coco
```

```
methode(2);
```

```
Param 1 = 2
```

```
methode(3, "salut", 1234);
```

Pas accepté par le compilateur :
Types incompatibles (**int** vs **String**)

Argument de taille variable (*vararg*)
toujours le dernier et 1 seule fois.

Arguments obligatoires précèdent
argument de taille variable (*vararg*).

Applications particulières

- **Cas de la méthode `printf`**

- `printf` écrit sur un flux de sortie (p.ex. la console). Le format d'affichage est spécifié à l'aide d'une *chaîne de caractère de format*.

```
public void printf(String format, Object... args);
```

- La chaîne `format` spécifie comment afficher chacun des éléments de `args` selon le format suivant

```
% [ flags ] [ largeur ] [ .precision ] conversion
```

- Exemple

```
System.out.println(Math.PI);      /* 3.141592653589793 */  
System.out.printf("%.2f\n", Math.PI);      /* 3.14 */  
System.out.printf("%04X\n", 165);      /* 00A5 */  
System.out.printf("Date = %02d/%02d/%04d\n",  
    jour, mois, annee); /* 30/01/2017 */
```

f = flottant

.2 = chiffres après virgule

x = entier, hexadécimal

4 = largeur

0 (flags) = "zero-padding"

d : entier, décimal

2 = largeur

0 = zero-padding

Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

Opérations

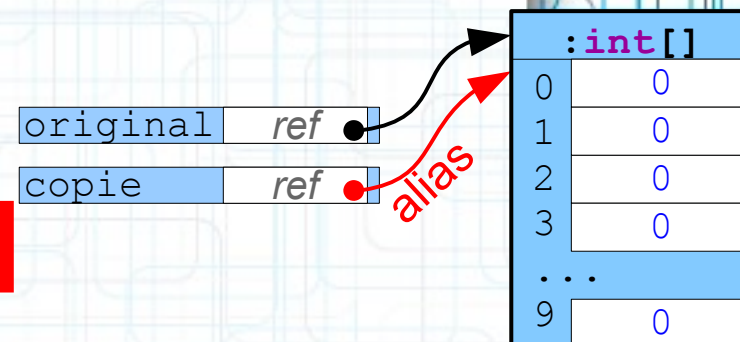
• Duplication de tableaux

- Il est parfois également nécessaire de dupliquer un tableau, i.e. créer un nouveau tableau dont la taille et le contenu sont identiques à un autre tableau.

- Peut-on procéder comme suit ?

```
int[] original = new int[10];
int[] copie = original;
```

NON



- Allouer un nouveau tableau et copier le contenu

```
int[] copie = new int[original.length];
for (int i = 0; i < original.length; i++)
    copie[i] = original[i];
```

```
int[] copie = new int[original.length];
System.arraycopy(original, 0, copie, 0, original.length);
```

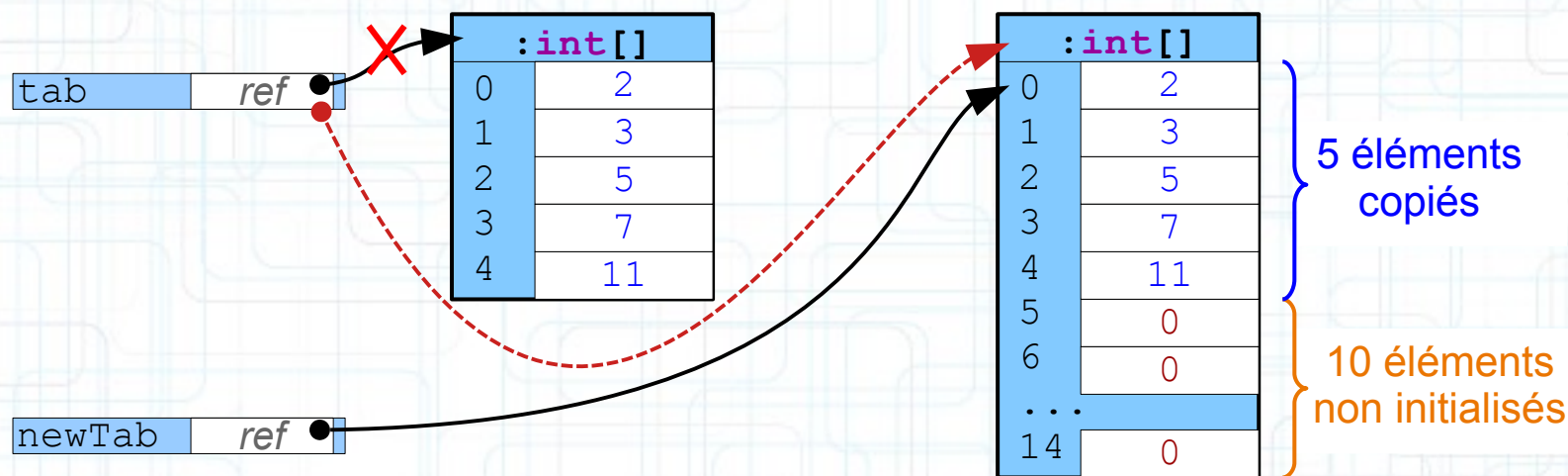
```
int[] copie = Arrays.copyOf(original, original.length);
```


Opérations

- **Agrandir un tableau**

- La taille d'un tableau est fixe, donc agrandir un tableau n'est pas possible. On peut cependant le remplacer par un tableau plus grand et de même contenu.

```
int[] tab = { 2, 3, 5, 7, 11 };  
int[] newTab = new int[15];  
System.arraycopy(tab, 0, newTab, 0, tab.length);  
tab = newTab;
```



- Cette approche fonctionne-t-elle avec un tableau de String ?

Opérations

- **Concaténation de deux tableaux**

- Une autre opération parfois nécessaire est la concaténation de deux tableaux. Il n'y a pas en Java d'opérateur permettant cette concaténation.

- Exemple

```
int[] tab1 = { 2, 3, 5, 7, 11 };  
int[] tab2 = { 13, 17, 19, 23 };  
int[] concat = new int[tab1.length + tab2.length];  
System.arraycopy(tab1, 0, concat, 0, tab1.length);  
System.arraycopy(tab2, 0, concat, tab1.length, tab2.length);
```

:int[5]	
0	2
1	3
2	5
3	7
4	11

:int[4]	
0	13
1	17
2	19
3	23

:int[9]	
0	2
1	3
2	5
3	7
4	11
5	13
6	17
7	19
8	23

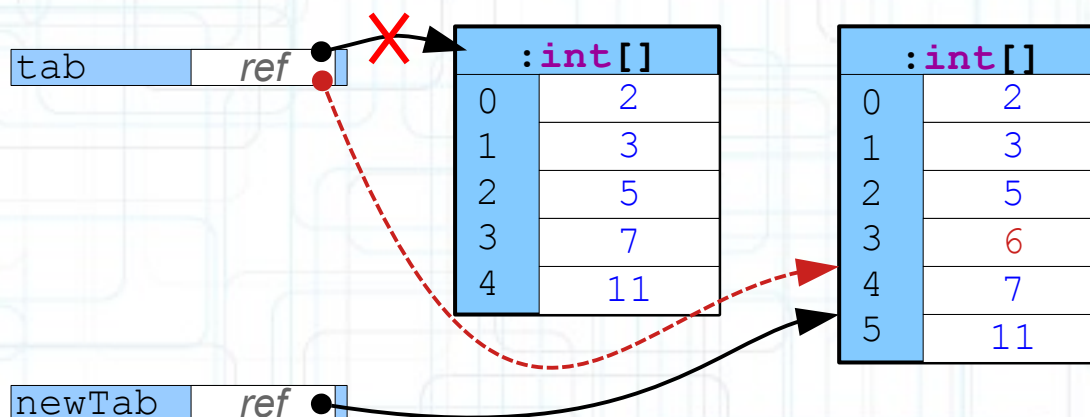


Opérations

- **Insertion d'un élément**

- L'insertion d'un élément requiert « l'agrandissement » du tableau, puis la copie des éléments précédant le point d'insertion, l'ajout du nouvel élément, et la copie des éléments suivant le point d'insertion.

```
int i = 3;           // point d'insertion
int newVal = 6;      // valeur à insérer
int[] tab = { 2, 3, 5, 7, 11 };
int[] newTab = new int[tab.length + 1];
System.arraycopy(tab, 0, newTab, 0, i);
System.arraycopy(tab, i, newTab, i + 1, tab.length - i);
tab = newTab;
```



Opérations

- **Méthodes utilitaires**

- La bibliothèque Java a été peu à peu enrichie de nombreuses méthodes de classe permettant d'effectuer des opérations utiles sur les tableaux telles que copie, tri, comparaison, test d'égalité, recherche dichotomique, etc.
- Ces méthodes sont fournies par la classe `java.util.Arrays`

```
static int binarySearch(T[] a, T key);  
static int compare(T[] a, T[] b);  
static T[] copyOf(T[] a, int newLength);  
static T[] copyOfRange(T[] a, int from, int to);  
static boolean equals(T[] a, T[] b);  
static void fill(T[] a, T val);  
static int mismatch(T[] a, T[] b);  
static void sort(T[]);  
static String toString(T[] a);  
static boolean deepEquals(T[] a, T[] b);  
static String deepToString(T[]);  
/* et bien d'autres ... */
```

ATTENTION. Dans ce contexte, le type `T` dénote l'un des types primitifs, le type `Object` ou un type générique.

Des variantes de ces méthodes sont fournies par surcharge pour chaque possibilité de `T`.

Opérations

- **Equivalent des accès aux listes en Python**

- indexation en partant de la fin ($1 \leq i \leq N$)

```
Python: a[-i]
Java:   a[a.length - i]
```

- tranche de *start* jusqu'à *stop*-1 ($0 \leq start < stop \leq N$)

```
Python: a[start:stop]
Java:   Arrays.copyOfRange(a, start, stop)
```

- tranche depuis *start* ($0 \leq start < N$)

```
Python: a[start:]
Java:   Arrays.copyOfRange(a, start, a.length)
```

- tranche jusqu'à *stop*-1 ($0 < stop \leq N$)

```
Python: a[:stop]
Java:   Arrays.copyOfRange(a, 0, stop)
```

- copie

```
Python: a[:]
Java:   Arrays.copyOf(a)
```

Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

Tableaux multi-dimensionnels

- **Type, déclaration, construction**

- $S = T []$ étant un type objet, il est possible de définir un tableau de type $S [] = T [] [] \dots$

- Syntaxe de la déclaration

```
nomType [ ]...[ ] nomVariable ;
```

Le nombre de paires de crochets détermine le nombre de dimensions du tableau.

- Syntaxe de la construction

```
new nomType [ nombreEltsDim1 ]...[ nombreEltsDimN ]
```

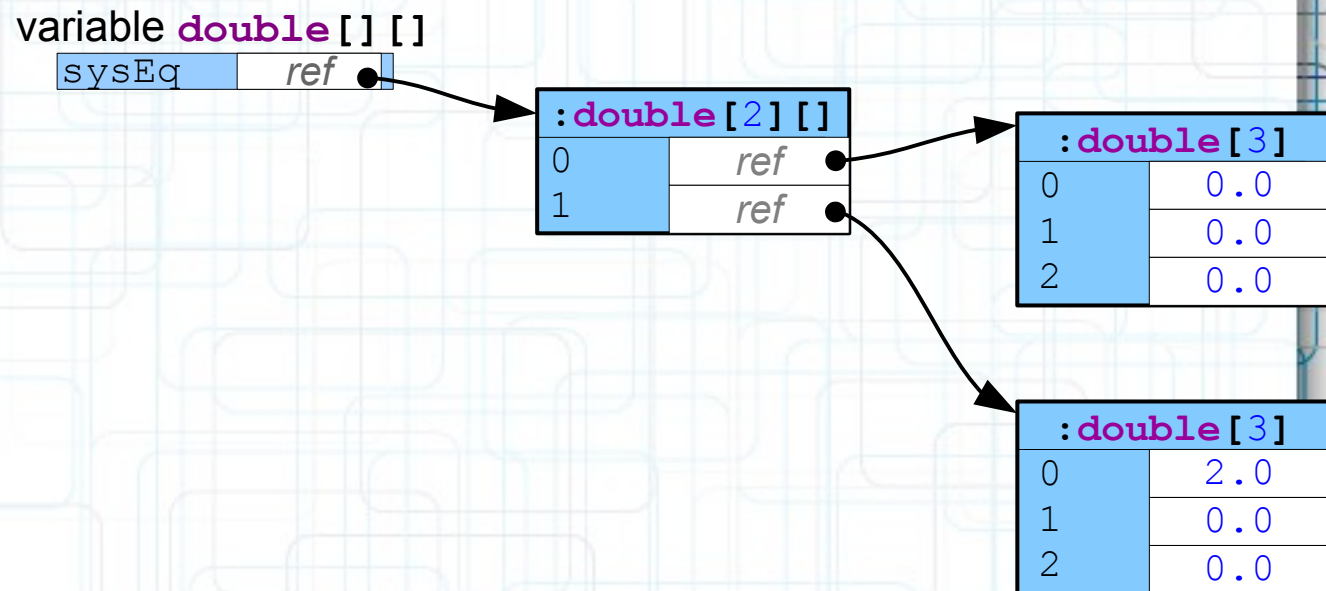
La taille de chaque dimension est spécifiée lors de la construction.

Note : La spécification de Java ne limite pas le nombre de dimensions. Cependant, le nombre de dimensions est typiquement limité à 255 dans l'implémentation de la JVM.

Tableaux multi-dimensionnels

- **En fait...**
 - ... les tableaux multi-dimensionnels **n'existent pas en Java !!**
 - Il s'agit en réalité de tableaux de tableaux.

```
double[][] sysEq = new double[2][3];  
/* Modification du coefficient de x  
   de la 2ème équation */  
sysEq[1][0] = 2.0;
```



Tableaux multi-dimensionnels

- **Initialisation**

- L'initialisation est possible lors de la déclaration, en utilisant plusieurs accolades imbriquées.

```
double[][] sysEq = {  
    { 1, -1, -1 },  
    { 2, -1, 1 }  
};
```

$$\begin{cases} x - y = -1 \\ 2x - y = 1 \end{cases}$$

variable **double**[][]
sysEq ref ●

:double[2][]	
0	ref ●
1	ref ●

:double[3]	
0	1.0
1	-1.0
2	-1.0

:double[3]	
0	2.0
1	-1.0
2	1.0

- Comment créeriez-vous une matrice 2 x 2 x 2 ?

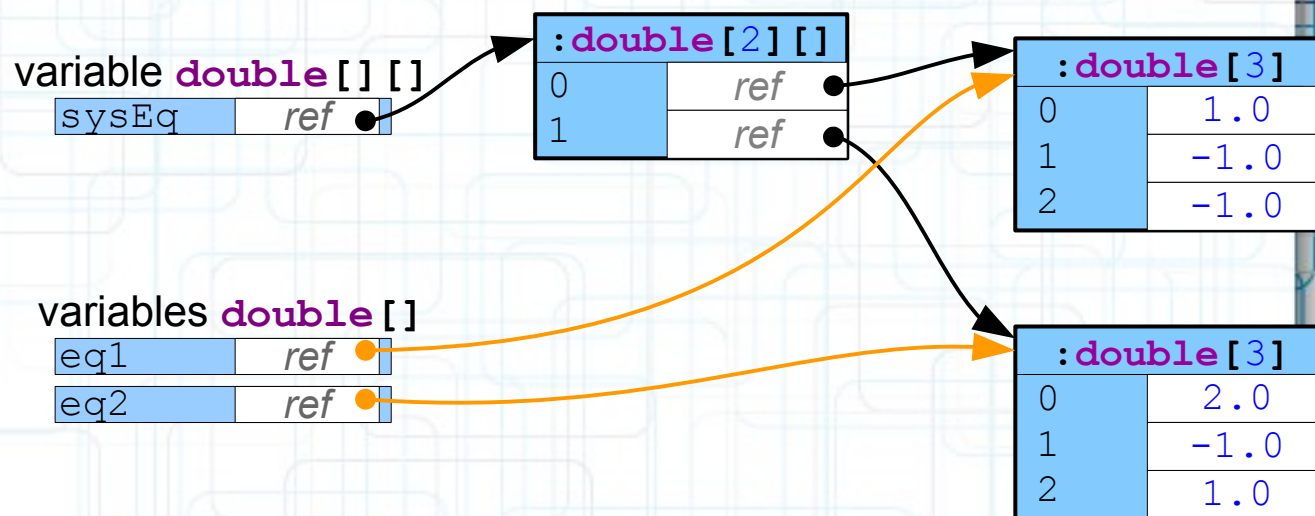
Tableaux multi-dimensionnels

- **Références vers les sous-tableaux**

- Il est possible de manipuler directement les références vers les « sous-tableaux » (en utilisant des alias).

```
double[][] sysEq = {
    { 1, -1, -1 },
    { 2, -1, 1 }
};
double[] eq1 = sysEq[0];
double[] eq2 = sysEq[1];
```

$$\begin{cases} x - y = -1 \\ 2x - y = 1 \end{cases}$$

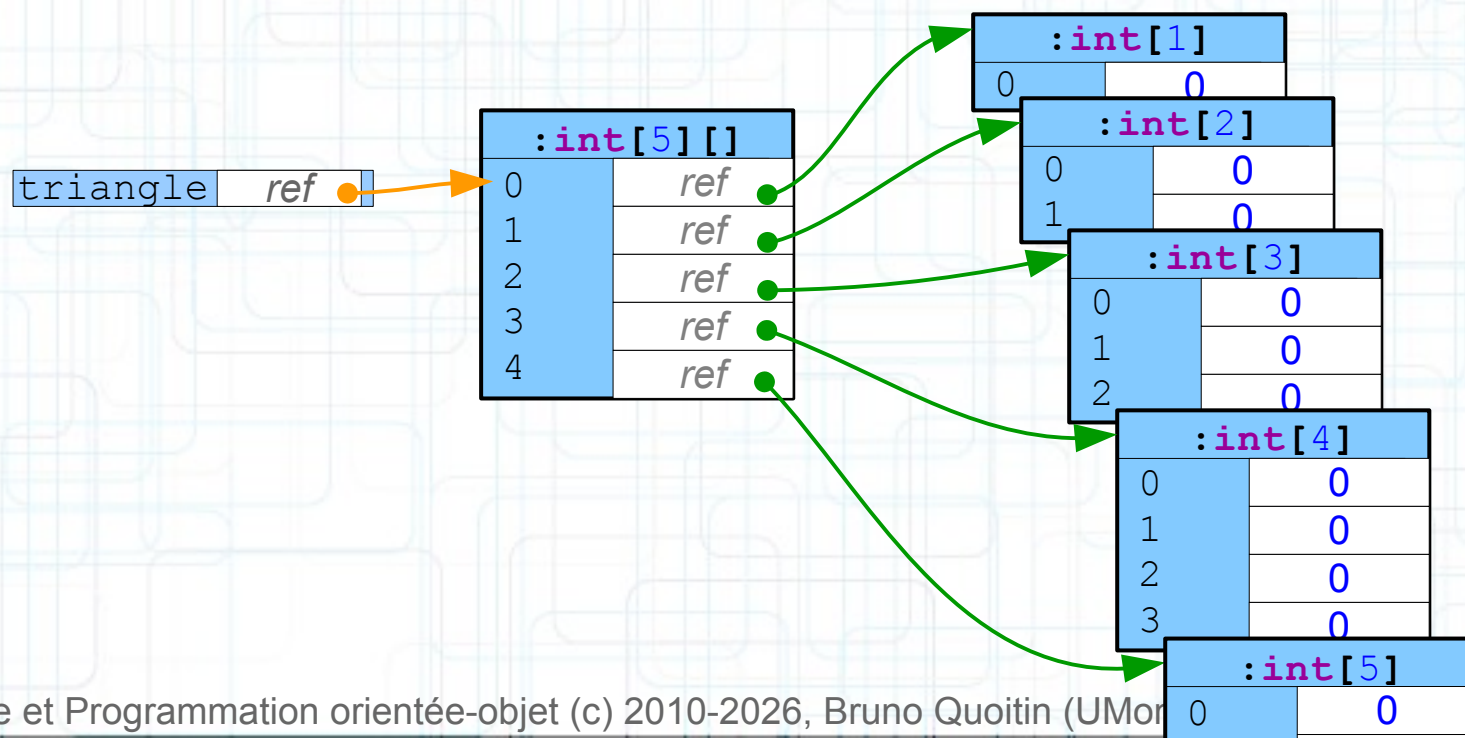
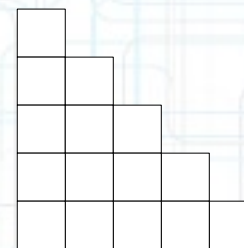


Tableaux multi-dimensionnels

- **Tableaux internes hétérogènes**

- Les tableaux internes d'un tableau de tableaux peuvent avoir des tailles différentes.

```
int[][] triangle = new int[5][];  
for (int i= 0; i < triangle.length; i++)  
    triangle[i] = new int[i+1];
```

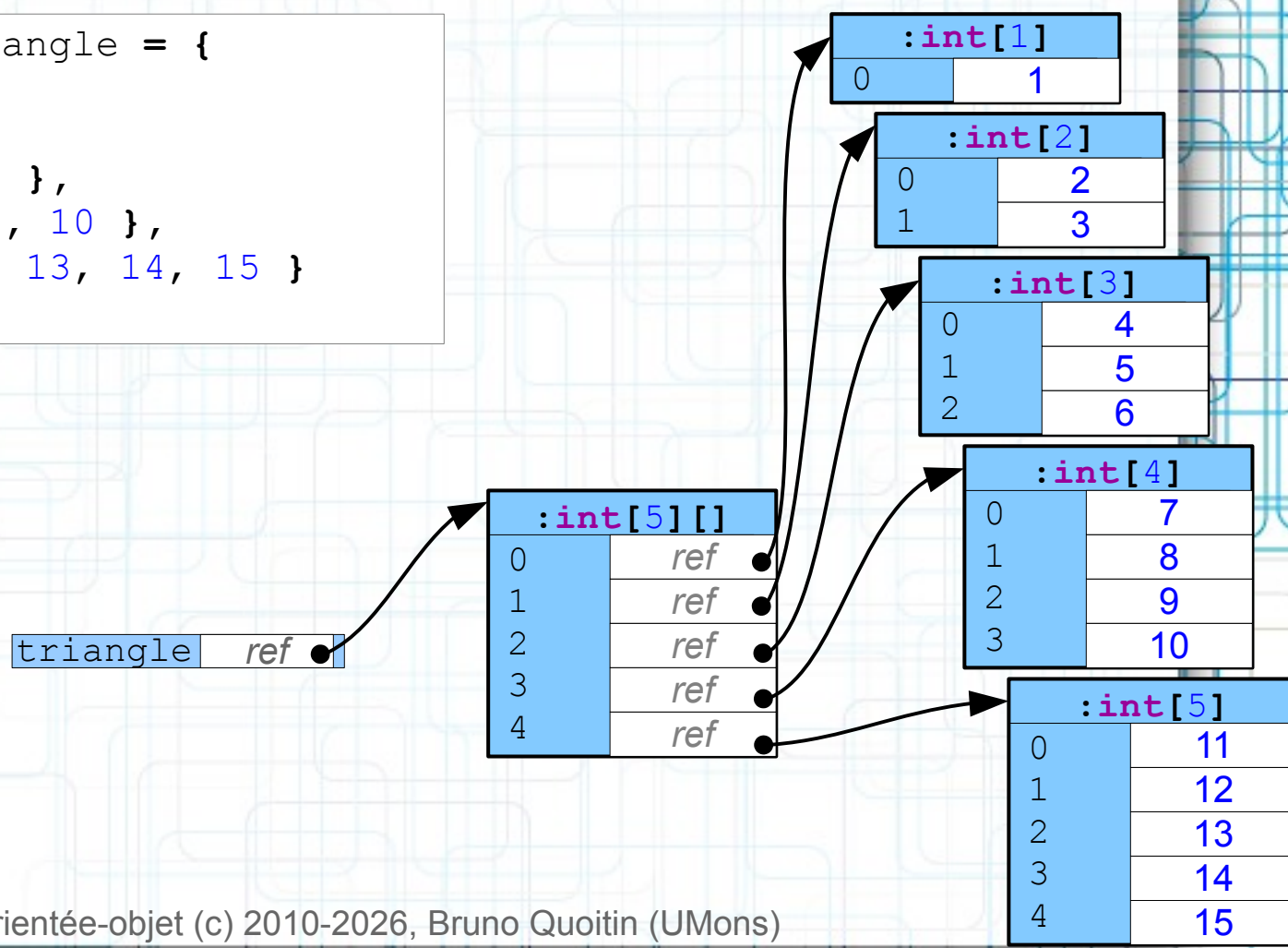


Tableaux multi-dimensionnels

• Initialisation

- Le nombre d'éléments fournis lors de l'initialisation peut être différent pour chaque dimension.

```
int[][] triangle = {
    { 1 },
    { 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9, 10 },
    { 11, 12, 13, 14, 15 }
};
```



Tableaux multi-dimensionnels

- **Exercice – Multiplication de matrices**
 - Donner une méthode de classe `multiplier` permettant d'effectuer la multiplication d'une matrice par une autre.
 - Les matrices sont modélisées par des tableaux à 2 dimensions de **double**.

Table des Matières

- **Introduction**
- **Type tableau**
 - Déclaration
 - Initialisation et instanciation
 - Opérateur d'indexation
 - Boucle for améliorée
- **Applications particulières**
- **Opérations sur les tableaux**
- **Plusieurs dimensions**
- **Application : combinaisons**

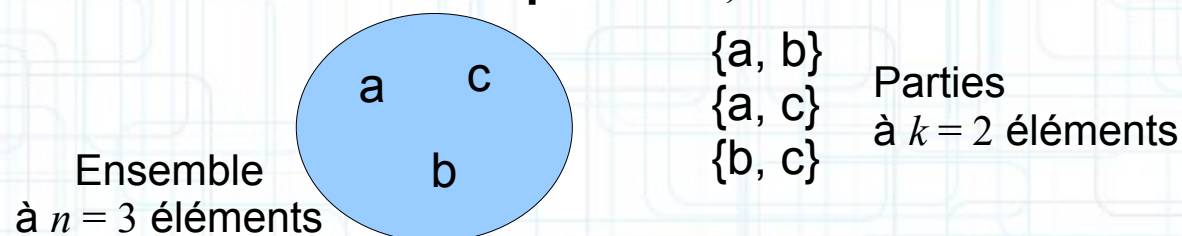
Application

- **Calcul de combinaisons**

- Objectif : déterminer le nombre de façons qu'il y a de prendre k éléments (sans remplacement) parmi n .
- Exemple : combien de combinaisons dans les cas suivants

- $n = 1, k = 1$
- $n = 2, k = 1$
- $n = 3, k = 2$

Illustration pour $n=3, k=2$



- Le nombre de combinaisons de k éléments pris dans un ensemble de n éléments peut-être calculé de la façon suivante

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Application

- **Combinaisons : récurrence**

- Un autre moyen de calculer le nombre de façons de prendre k éléments parmi n consiste à utiliser la **relation de récurrence** suivante (« *relation de Pascal* »).

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}, \quad \forall 0 < k < n$$

- Les cas de base sont

$$\binom{n}{0} = 1 \quad \binom{n}{n} = 1$$

$\underline{k=0} \qquad \underline{k=n}$

$$\forall k > n, \quad \binom{n}{k} = 0$$

Application

- **Combinaisons : récurrence**

- Un algorithme récursif vient « immédiatement » à l'esprit.

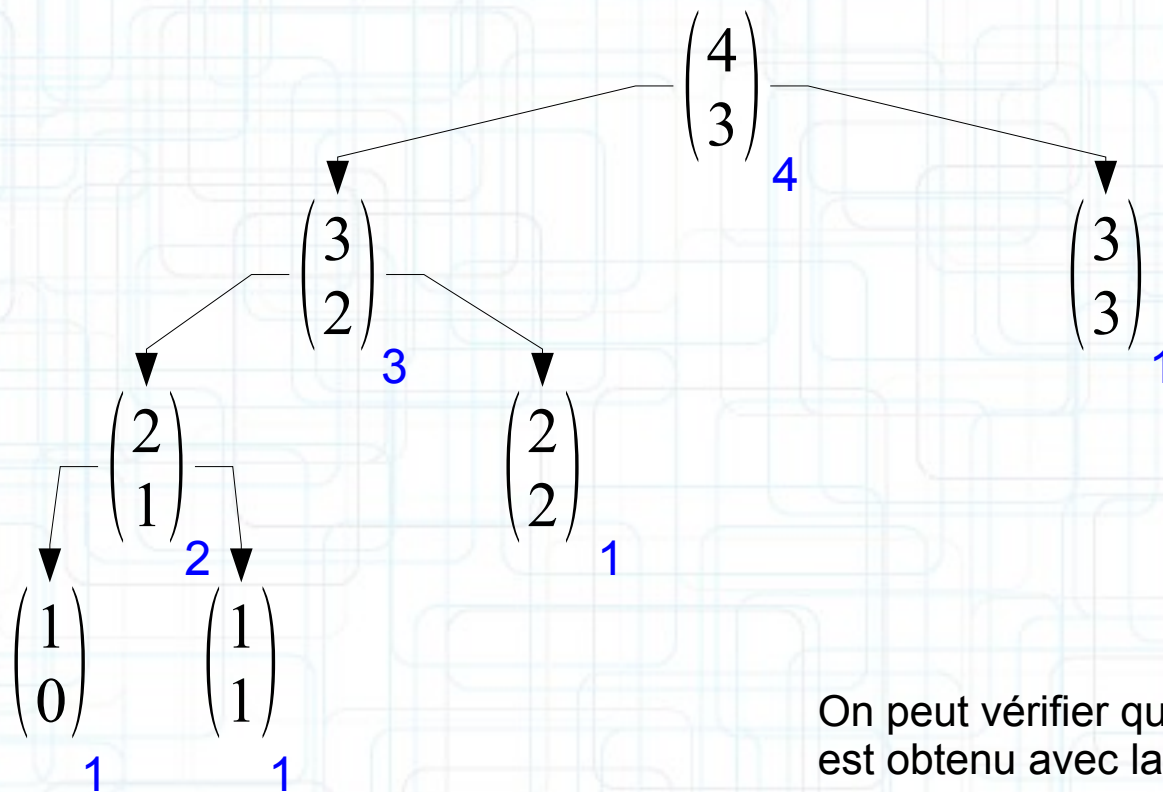
```
public static int comb(int n, int k)
{
    if ((k == 0) || (k == n))
        return 1;
    if (k > n)
        return 0;
    return comb(n-1, k-1) + comb(n-1, k);
}
```

- Il s'agit d'une **double récursion**. Le problème de cette approche est qu'elle peut nécessiter un très grand nombre d'appels récursifs.

Application

- Combinaisons : récurrence**

- Exemple : calcul du nombre de façons de prendre $k=3$ éléments parmi $n=4$.



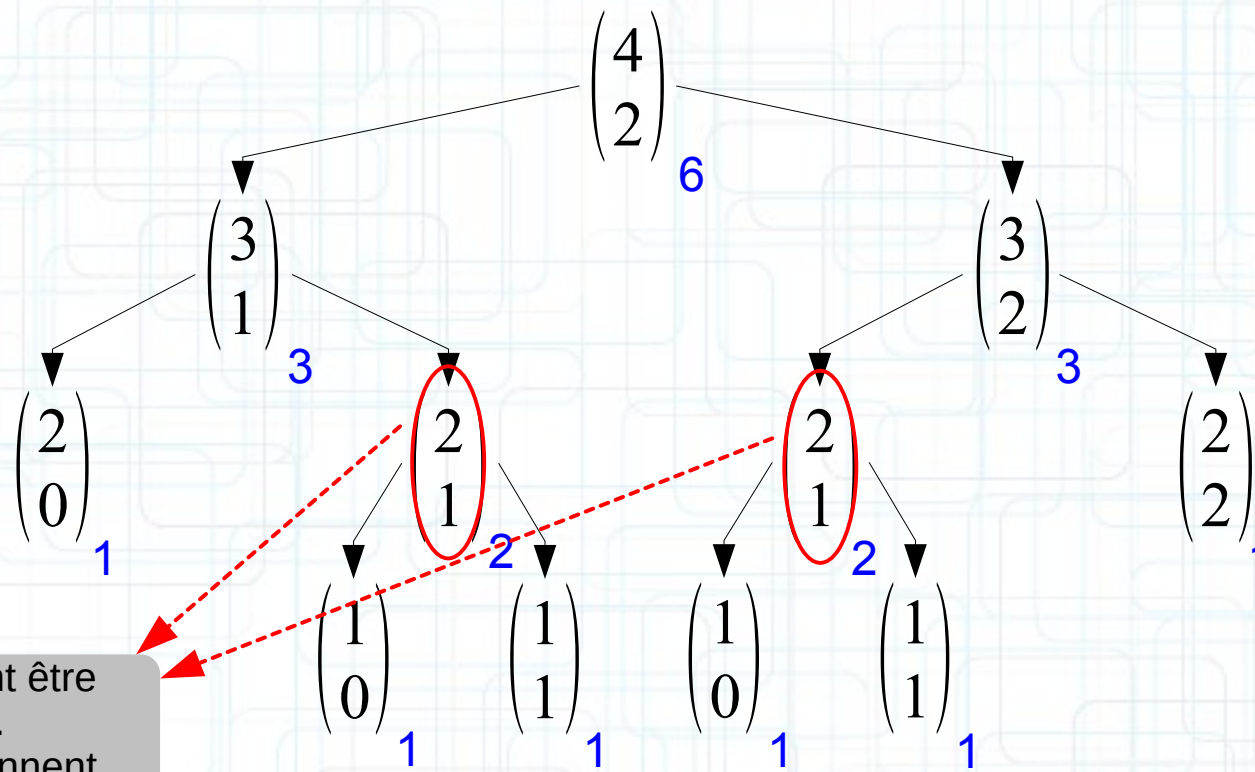
On peut vérifier que le même résultat est obtenu avec la factorielle

$$\binom{4}{3} = \frac{4!}{3!(4-3)!} = \frac{24}{6 \times 1} = 4$$

Application

- Combinaisons : récurrence**

- Exemple : calcul du nombre de façons de prendre $k=2$ éléments parmi $n=4$.



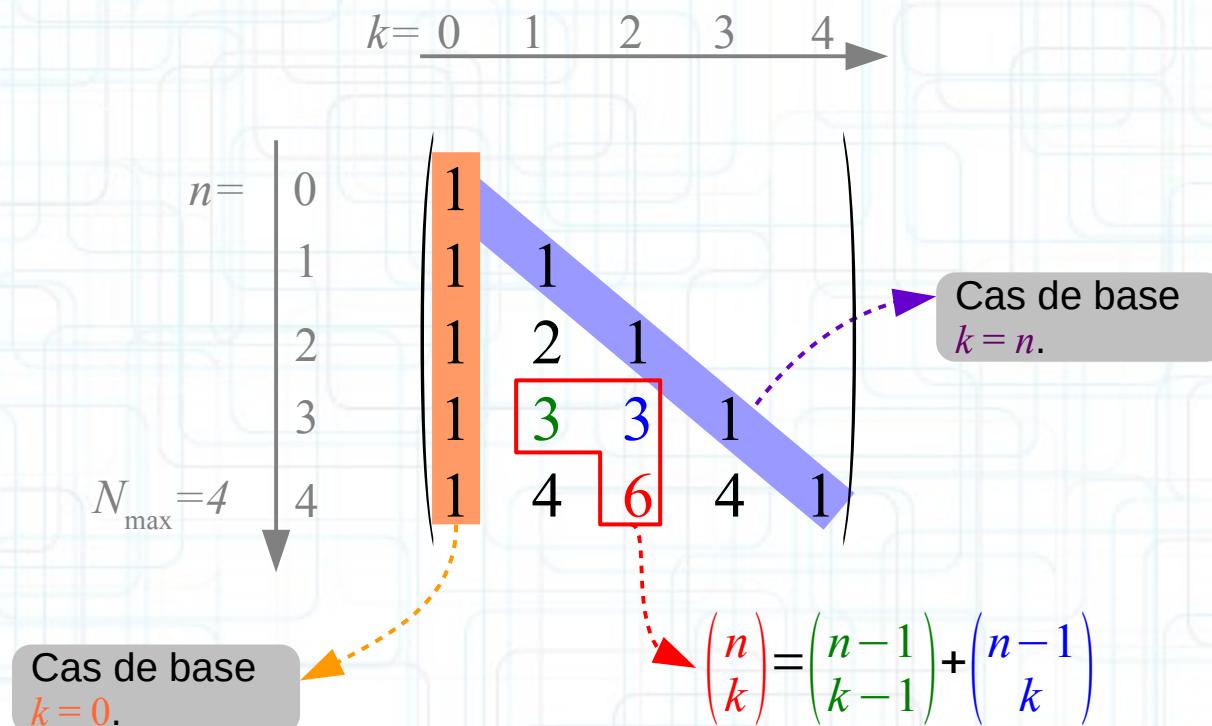
Certains cas peuvent être traités plusieurs fois. Lorsque k et n deviennent grands, le nombre de calculs dupliqués est important !

$$\binom{4}{2} = \frac{4!}{2!(4-2)!} = \frac{24}{2 \times 2} = 6$$

Application

- **Combinaisons : mémorisation**

- On peut calculer **de façon plus efficace** le nombre de combinaisons pour tous les couples (n,k) tels que $0 \leq k \leq n \leq N_{\max}$, en calculant la matrice triangulaire suivante⁽¹⁾.

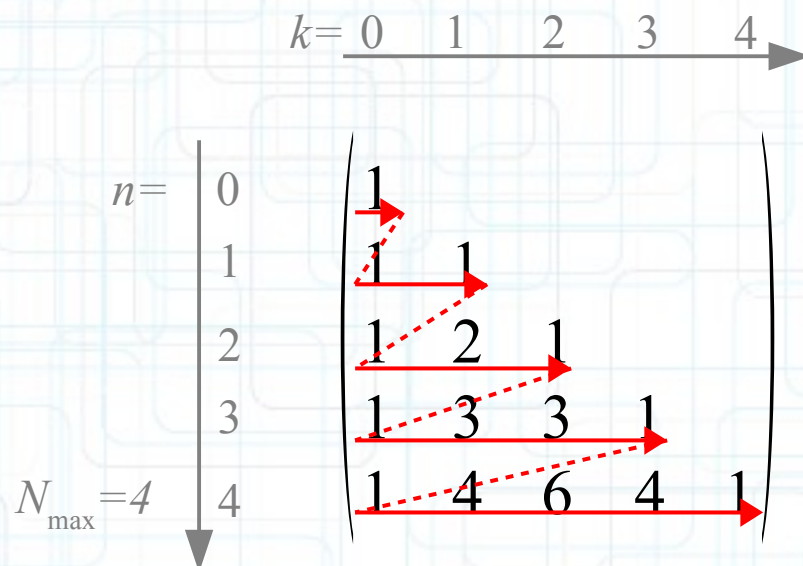


⁽¹⁾ appelée « triangle de Pascal »

Application

- Combinaisons : mémorisation**

- On peut calculer **de façon plus efficace** le nombre de combinaisons pour tous les couples (n,k) tels que $0 \leq k \leq n \leq N_{\max}$, en calculant la matrice triangulaire suivante.



Application

- **Combinaisons : mémorisation**

- Construction de la matrice triangulaire des combinaisons, pour $0 \leq k \leq n \leq N_{\max}$

```
final int N_MAX = 7;  
int[][] comb = new int[N_MAX + 1][];-----  
for (int n = 0; n < comb.length; n++) {  
    comb[n] = new int[n+1];  
    for (int k = 0; k <= n; k++)  
    {  
        if ((k == n) || (k == 0))  
            comb[n][k] = 1; /* cas de base */  
        else  
            comb[n][k] = comb[n-1][k-1] +  
                        comb[n-1][k]; /* cas récursif */  
    }  
}
```

index extérieur pour n

index intérieur pour k

$$\binom{n}{0} = \binom{n}{n} = 1$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Application

- **Combinaisons : mémoïsation**

- Affichage du tableau résultant.

```
for (int i= 0; i < comb.length; i++) {
    for (int j= 0; j < comb[i].length; j++)
        System.out.printf("%2d", comb[i][j]);
    System.out.println();
}
```

	k							
	0	1	2	3	4	5	6	7
0	1							
1	1	1						
2	1	2	1					
3	1	3	3	1				
4	1	4	6	4	1			
5	1	5	10	10	5	1		
6	1	6	15	20	15	6	1	
7	1	7	21	35	35	21	7	1

cas $N_{\max} = 7$